

# 1

## Why Quantum Computing?

*Nature isn't classical, dammit,  
and if you want to make a simulation of nature,  
you'd better make it quantum mechanical.*

Richard Feynman [5]

In his 1982 paper “Simulating Physics with Computers,” Richard Feynman, 1965 Nobel Laureate in Physics, said he wanted to “talk about the possibility that there is to be an *exact* simulation, that the computer will do *exactly* the same as nature.” He then went on to make the statement above, asserting that nature doesn’t especially make itself amenable for computation via classical binary computers.

In this chapter we begin to explore how quantum computing is different from classical computing. Classical computing is what drives smartphones, laptops, Internet servers, mainframes, high performance computers, and even the processors in automobiles.

We examine several use cases where quantum computing may someday help us solve problems that are today intractable using classical methods on classical computers. This is to motivate you to learn about the underpinnings and details of quantum computers I discuss throughout the book.

No single book on this topic can be complete. The technology and potential use cases are moving targets as we innovate and create better hardware and software. My goal here is

to prepare you to delve more deeply into the science, coding, and applications of quantum computing.

## Topics covered in this chapter

|     |                                                   |    |
|-----|---------------------------------------------------|----|
| 1.1 | The mysterious quantum bit . . . . .              | 2  |
| 1.2 | I'm awake! . . . . .                              | 4  |
| 1.3 | Why quantum computing is different . . . . .      | 7  |
| 1.4 | Applications to artificial intelligence . . . . . | 9  |
| 1.5 | Applications to financial services . . . . .      | 15 |
| 1.6 | What about cryptography? . . . . .                | 18 |
| 1.7 | Summary . . . . .                                 | 21 |

## 1.1 The mysterious quantum bit

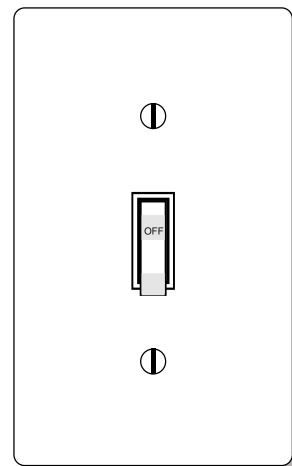
Suppose I am standing in a room with a single overhead light and a switch that turns the light on or off. This is just a normal switch, and so I can't dim the light. It is either fully on or fully off. I can change it at will, but this is the only thing I can do to it. There is a single door to the room and no windows. When the door is closed I cannot see any light.

I can stay in the room or I may leave it. The light is always on or off based on the position of the switch.

Now I'm going to do some rewiring. I'm replacing the switch with one that is in another part of the building. I can't see the light at all but, once again, its being on or off is determined solely by the two positions of the switch.

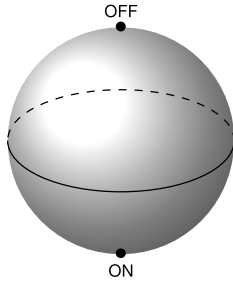
If I walk to the room with the light and open the door, I can see whether it is lit or dark. I can walk in and out of the room as many times as I want and the status of the light is still determined by that remote switch being on or off. This is a "classical" light.

Now let's imagine a *quantum* light and switch, which I'll call a "qu-light" and "qu-switch," respectively.



## 1.1. The mysterious quantum bit

---



When I walk into the room with the qu-light it is always on or off, just like before. The qu-switch is unusual in that it is shaped like a sphere with the topmost point (the “north pole”) being OFF and the bottommost (the “south pole”) being ON. There is a line etched around the middle.

The interesting part happens when I cannot see the qu-light, when I am in the other part of the building with the qu-switch.

I control the qu-switch by placing my index finger on the qu-switch sphere. If I place my finger on the north pole, the qu-light is definitely off. If I put it on the south, the qu-light is definitely on. You can go into the room and check. You will always get these results.

If I move my finger anywhere else on the qu-switch sphere, the qu-light may be on or off when you check. If you do not check, the qu-light is in an indeterminate state. It is not dimmed, it is not on or off, it just exists with some probability of being on or off when seen. This is unusual!

The moment you open the door and see the qu-light, the indeterminacy is removed. It will be on or off. Moreover, if I had my finger on the qu-switch, the finger would be forced to one or other of the poles corresponding to the state of the qu-light when it was seen.

The act of observing the qu-light forced it into either the on or off state. I don’t have to see the qu-light fixture itself. If I open the door a tiny bit, enough to see if any light is shining or not, that is enough.

If I place a video camera in the room with the qu-light and watch it when I try to place my finger on the qu-switch, it behaves just like a normal switch. I will be prevented from touching the qu-switch at anywhere other than the top or bottom. Since I’m making up this example, assume some sort of force field keeps me away from anywhere but the poles!

If you or I are not observing the qu-light in any way, does it make a difference where I touch the qu-switch? Will touching it in the northern or southern hemisphere influence whether it will be on or off when I observe the qu-light?

Yes. Touching it closer to the north pole or the south pole will make the probability of the qu-light being off or on, respectively, be higher. If I put my finger on the circle between the poles, the equator, the probability of the light being on or off will be exactly 50 – 50.

What I just described is called a *two-state quantum system*. When it is not being observed, the qu-light is in a *superposition* of being on and off. We explore superposition in [section 7.1](#).

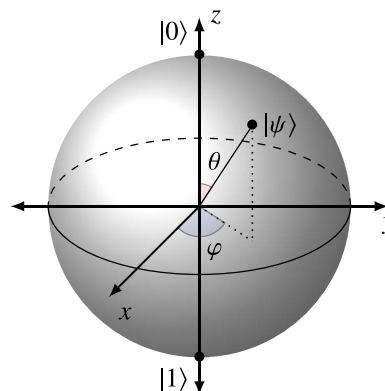
While this may seem bizarre, evidently nature really works this way. Electrons have a property called “spin” and with this they are two-state quantum systems. The photons that make

up light itself are two-state quantum systems. We return to this in [section 11.3](#) when we look at polarization (as in Polaroid<sup>®</sup> sunglasses).

More to the point of this book, however, a *quantum bit*, more commonly known as a *qubit*, is a two-state quantum system. It extends and complements the classical computing notion of bit, which can only be 0 or 1. The qubit is the basic information unit in quantum computing.

This book is about how we manipulate qubits to solve problems that currently appear to be intractable using just classical computing. It seems that just sticking to 0 or 1 will not be sufficient to solve some problems that would otherwise need impractical amounts of time or memory.

With a qubit, we replace the terminology of on or off, 1 or 0, with  $|1\rangle$  and  $|0\rangle$ , respectively. Instead of qu-lights, it's qubits from now on.



In the diagram on the right, the position of your finger on the qu-switch is now indicated by two angles,  $\theta$  and  $\varphi$ . The picture itself is called a Bloch sphere and is a standard representation of a qubit, as we shall see in [section 7.5](#).

## 1.2 I'm awake!

What if we could do chemistry inside a computer instead of in a test tube or beaker in the laboratory? What if running a new experiment was as simple as running an app and having it complete in a few seconds?

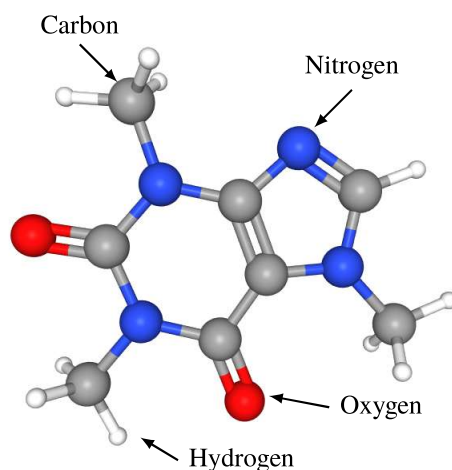
For this to really work, we would want it to happen with full *fidelity*. The atoms and molecules as modeled in the computer should behave **exactly** like they do in the test tube. The chemical reactions that happen in the physical world would have precise computational analogs. We would need a fully faithful simulation.

If we could do this at scale, we might be able to compute the molecules we want and need. These might be for new materials for shampoos or even alloys for cars and airplanes. Perhaps we could more efficiently discover medicines that are customized to your exact physiology. Maybe we could get better insight into how proteins fold, thereby understanding their function, and possibly creating custom enzymes to positively change our body chemistry.

## 1.2. I'm awake!

---

Is this plausible? We have massive supercomputers that can run all kinds of simulations. Can we model molecules in the above ways today?



Let's start with  $C_8H_{10}N_4O_2$  – 1,3,7-Trimethylxanthine. This is a very fancy name for a molecule which millions of people around the world enjoy every day: **caffeine**. An 8 ounce cup of coffee contains approximately 95 mg of caffeine, and this translates to roughly  $2.95 \times 10^{20}$  molecules. Written out, this is

295,000,000,000,000,000,000 molecules.

A 12 ounce can of a popular cola drink has 32 mg of caffeine, the diet version has 42 mg, and energy drinks often have about 77 mg. [11]

### Question 1.2.1

How many molecules of caffeine do you consume a day?

These numbers are large because we are counting physical objects in our universe, which we know is very big. Scientists estimate, for example, that there are between  $10^{49}$  and  $10^{50}$  atoms in our planet alone. [4]

To put these values in context, one thousand =  $10^3$ , one million =  $10^6$ , one billion =  $10^9$ , and so on. A gigabyte of storage is one billion bytes, and a terabyte is  $10^{12}$  bytes.



**To learn more**

Quantum chemistry is not an area of science in which you can say a few words and easily make clear how quantum computers might eventually be used to compute molecular properties and protein folding configurations, for example. Nevertheless, the caffeine example above is an example of *quantum simulation*.

For an excellent survey of the history and state of the art of quantum computing applied to chemistry as of 2019, see Cao et al. [2] For the specific problem of understanding how to scale quantum simulations of molecules and the crossover from High Performance Computers (HPC), see Kandala et al. [10]

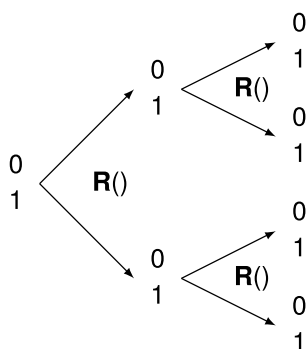
## 1.3 Why quantum computing is different

I can write a little app on a classical computer that can simulate a coin flip. This might be for my phone or laptop.

Instead of heads or tails, let's use 1 and 0. The routine, which I call **R**, starts with one of those values and randomly returns one or the other. That is, 50% of the time it returns 1 and 50% of the time it returns 0. We have no knowledge whatsoever of how **R** does what it does. When you see “**R**,” think “random.”

This is called a “fair flip.” It is not weighted to slightly prefer one result or the other. Whether we can produce a truly random result on a classical computer is another question. Let's assume our app is fair.

If I apply **R** to 1, half the time I expect that same value and the other half 0. The same is true if I apply **R** to 0. I'll call these applications **R**(1) and **R**(0), respectively.



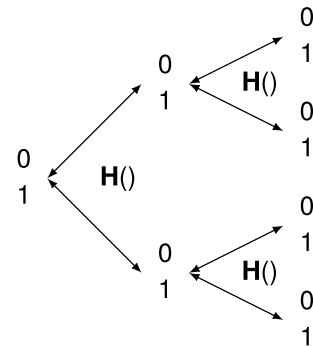
If I look at the result of **R**(1) or **R**(0), there is no way to tell if I started with 1 or 0. This is just as in a secret coin flip where I can't tell whether I began with heads or tails just by looking at how the coin has landed. By “secret coin flip,” I mean that someone else does it and I can see the result, but I have no knowledge of the mechanics of the flip itself or the starting state of the coin.

If **R**(1) and **R**(0) are randomly 1 and 0, what happens when I apply **R** twice?

I write this as  $\mathbf{R}(\mathbf{R}(1))$  and  $\mathbf{R}(\mathbf{R}(0))$ . It's the same answer: random result with an equal split. The same thing happens no matter how many times we apply  $\mathbf{R}$ . The result is random and we can't reverse things to learn the initial value. In the language of section 4.1,  $\mathbf{R}$  is not *invertible*.

Now for the quantum version. Instead of  $\mathbf{R}$ , I use  $\mathbf{H}$ , which we learn about in section 7.6. It too returns 0 or 1 with equal chance but it has two interesting properties:

1. It is reversible. Though it produces a random 1 or 0 starting from either of them, we can always go back and see the value with which we began.
2. It is its own reverse (or *inverse*) operation. Applying it two times in a row is the same as having done nothing at all.



There is a catch, though. You are not allowed to look at the result of what  $\mathbf{H}$  does if you want to reverse its effect.

If you apply  $\mathbf{H}$  to 0 or 1, peek at the result, and apply  $\mathbf{H}$  again to that, it is the same as if you had used  $\mathbf{R}$ . If you observe what is going on in the quantum case at the wrong time, you are right back at strictly classical behavior.

To summarize using the coin language: if you flip a quantum coin and then **don't look at it**, flipping it again will yield the heads or tails with which you started. If you do look, you get classical randomness.

### Question 1.3.1

Compare this behavior with that of the qu-switch and qu-light in section 1.1.

A second area where quantum is different is in how we can work with simultaneous values. Your phone or laptop uses bytes as the individual units of memory or storage. That's where we get phrases like "megabyte," which means one million bytes of information.

A byte is further broken down into eight bits, which we've seen before. Each bit can be 0 or 1. Doing the math, each byte can represent  $2^8 = 256$  different numbers composed of eight 0s or 1s, but it can only hold *one value at a time*.

Eight qubits can represent all 256 values *at the same time*.

This is through superposition, but also through *entanglement*, the way we can tightly tie together the behavior of two or more qubits. This is what gives us the (literally) exponential

growth in the amount of working memory that we saw with a quantum representation of caffeine in [section 1.2](#). We explore entanglement in [section 8.2](#).

## 1.4 Applications to artificial intelligence

Artificial intelligence and one of its subsets, machine learning, are extremely broad collections of data-driven techniques and models. They are used to help find patterns in information, learn from the information, and automatically perform more “intelligently.” They also give humans help and insight that might have been difficult to get otherwise.

Here is a way to start thinking about how quantum computing might be applicable to large, complicated, computation-intensive systems of processes such as those found in AI and elsewhere. These three cases are in some sense the “small, medium, and large” ways quantum computing might complement classical techniques:

1. There is a single mathematical computation somewhere in the middle of a software component that might be sped up via a quantum algorithm.
2. There is a well described component of a classical process that could be replaced with a quantum version.
3. There is a way to avoid the use of some classical components entirely in the traditional method because of quantum, or the entire classical algorithm can be replaced by a much faster or more effective quantum alternative.

As I write this, quantum computers are not “big data” machines. This means you cannot take millions of records of information and provide them as input to a quantum calculation. Instead, quantum may be able to help where the number of inputs are modest but the computations “blow up” as you start examining relationships or dependencies in the data. Quantum, with its exponentially growing working memory, as we saw in the caffeine example in [section 1.2](#), may be able to control and work with the blow up. (See [section 2.7](#) for a discussion of exponential growth.)

In the future, however, quantum computers may be able to input, output, and process much more data. Even if it is just theoretical now, it makes sense to ask if there are quantum algorithms that can be useful in AI someday.

Let’s look at some data. I’m a big baseball fan, and baseball has a lot of statistics associated with it. The analysis of this even has its own name: “sabermetrics.”

| <b>Year</b> | <b>GP</b> | <b>AB</b> | <b>R</b> | <b>H</b> | <b>2B</b> | <b>3B</b> | <b>HR</b> | <b>RBI</b> | <b>BB</b> | <b>SO</b> |
|-------------|-----------|-----------|----------|----------|-----------|-----------|-----------|------------|-----------|-----------|
| 2019        | 136       | 470       | 105      | 136      | 27        | 2         | 41        | 101        | 110       | 111       |
| 2018        | 162       | 587       | 94       | 156      | 25        | 1         | 46        | 114        | 74        | 173       |
| 2017        | 152       | 542       | 73       | 132      | 24        | 0         | 29        | 92         | 41        | 145       |
| 2016        | 140       | 490       | 84       | 123      | 26        | 5         | 27        | 70         | 50        | 109       |
| 2015        | 162       | 634       | 66       | 172      | 32        | 4         | 25        | 83         | 26        | 108       |
| 2014        | 148       | 545       | 110      | 153      | 35        | 1         | 29        | 79         | 74        | 144       |

where

**GP** = Games Played

**AB** = At Bats

**R** = Runs scored

**H** = Hits

**2B** = 2 Base hits (doubles)

**3B** = 3 Base hits (triples)

**HR** = Home Runs

**RBI** = Runs Batted In

**BB** = Bases on Balls (walks)

**SO** = Strike Outs

**Figure 1.1: Baseball player statistics by year**

Suppose I have a table of statistics for a baseball player given by year as shown in [Figure 1.1](#). We can make this look more mathematical by creating a matrix of the same data.

$$\begin{bmatrix} 2019 & 136 & 470 & 105 & 136 & 27 & 2 & 41 & 101 & 110 & 111 \\ 2018 & 162 & 587 & 94 & 156 & 25 & 1 & 46 & 114 & 74 & 173 \\ 2017 & 152 & 542 & 73 & 132 & 24 & 0 & 29 & 92 & 41 & 145 \\ 2016 & 140 & 490 & 84 & 123 & 26 & 5 & 27 & 70 & 50 & 109 \\ 2015 & 162 & 634 & 66 & 172 & 32 & 4 & 25 & 83 & 26 & 108 \\ 2014 & 148 & 545 & 110 & 153 & 35 & 1 & 29 & 79 & 74 & 144 \end{bmatrix}$$

Given such information, we can manipulate it using machine learning techniques to make predictions about the player's future performance or even how other similar players may do. These techniques make use of the matrix operations we discuss in [chapter 5](#).

There are 30 teams in Major League Baseball in the United States. With their training and feeder “minor league” teams, each major league team may each have more than 400 players throughout their systems. That would give us over 12,000 players, each with their complete player histories. There are more statistics than I have listed, so we can easily get greater than 100,000 values in our matrix.

## 1.4. Applications to artificial intelligence

---

In the area of entertainment, it's hard to make an estimate of how many movies exist, but it is well above 100,000. For each movie, we can list features such as whether it is a comedy or a drama or a romance or an action film, who each of the actors are, who each of the directorial and production staff are, geographic locations shown in the film, languages used, and so on. There are hundreds of such features and *millions of people who have watched the films!*

For each person, we can also add features such as whether they like or dislike a kind of movie, actor, scene location, or director. Using all this information, which film should I recommend to you on a Saturday night in December based on what you and people similar to you like?

Think of each feature or each baseball player or film as a dimension. While you may think of two and three dimensions in nature, in AI we might have thousands or millions of dimensions.

Matrices as above for AI can grow to millions of rows and entries. How can we make sense of them to get insights and see patterns? Aside from manipulating that much information, can we even eventually do the math on classical computers quickly and accurately enough?

While it was originally thought that quantum algorithms might offer exponential improvements of such classical recommender systems, a 2019 “quantum-inspired algorithm” by Ewin Tang showed a classical method to gain such a huge improvement. [17] An example of being exponentially faster is doing something in 6 days instead of  $10^6 = 1$  million days. That's approximately 2,740 years.

Tang's work is a fascinating example of the interplay of progress in both classical and quantum algorithms. People who develop algorithms for classical computing look to quantum computing, and vice versa. Also, any particular solution to a problem may include classical and quantum components.

Nevertheless, many believe that quantum computing will show very large improvements for some matrix computations. One such example is the **HHL** algorithm, whose abbreviation comes from the first letters of the last names of its authors, Aram W. Harrow, Avinatan Hassidim, and Seth Lloyd. This is also an example of [case number 1](#) above.

Algorithms such as these may find use in fields as diverse as economics and computational fluid dynamics. They also place requirements on the structure and density of the data and may use properties such as the [condition number](#) we discuss in [subsection 5.7.6](#).

### To learn more

When you complete this book you will be equipped to read the original paper describing the **HHL** algorithm and more recent surveys about how to apply quantum computing to [linear algebraic](#) problems. [7]

An important problem in machine learning is classification. In its simplest form, a *binary classifier* separates items into one of two categories, or buckets. Depending on the definitions of the categories, it may be more or less easy to do the classification.

Examples of binary categories include:

- book you like **or** book you don't like
- comedy movie **or** dramatic movie
- gluten-free **or** not gluten-free
- fish dish **or** chicken dish
- UK football team **or** Spanish football team
- hot sauce **or** very hot sauce
- cotton shirt **or** permanent press shirt
- open source **or** proprietary
- spam email **or** valid email
- American League baseball team **or** National League team

The second example of distinguishing between comedies and dramas may not be well designed since there are movies that are both.

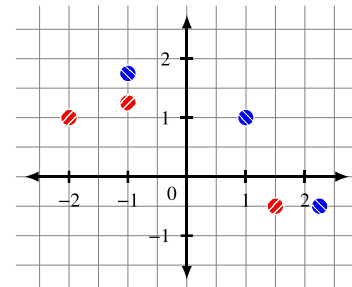
Mathematically, we can imagine taking some data as input and classifying it as either +1 or -1. We take a reasonably large set of data and label it by hand as either being a +1 or -1. We then *learn* from this *training set* how to classify future data.

Machine learning binary classification algorithms include random forest, k-nearest neighbor, decision tree, neural networks, naive Bayes classifiers, and support vector machines (SVMs).

In the training phase, we are given a list of pre-classified objects (books, movies, proteins, operating systems, baseball teams, etc.). We then use the above algorithms to learn how to put a new object in one bucket or another.

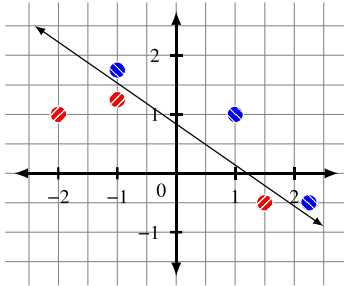
The SVM is a straightforward approach with a clear mathematical description. In the two-dimensional case, we try to draw a line that separates the objects (represented by points in the plot to the right) into one category or the other.

The line should maximize the gap between the sets of objects.



## 1.4. Applications to artificial intelligence

---



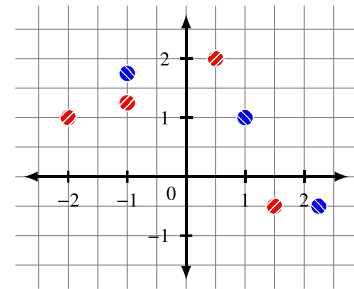
On the left is an example of a line that separates the red points below from the blue points above the line.

Given a new point, we plot it and determine whether it is above or below the line. That will classify it as blue or red, respectively.

Suppose we know that the point is correctly classified with those above the line. We accept that and move on.

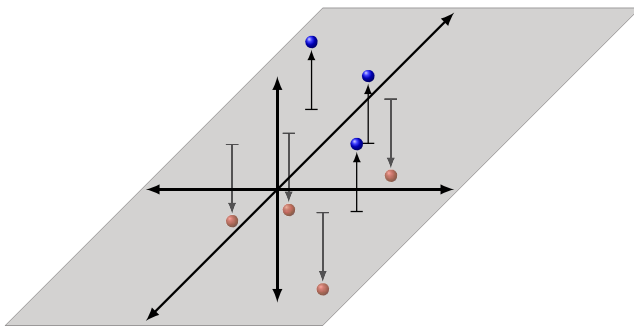
If the point is misclassified, we add the point to the training set and try to compute a new and better line. This may not be possible.

In the plot to the right, I added a new red point above the line close to 2 on the vertical axis. With this extra point, there is no line we can compute to separate the points.



Had we represented the objects in three dimensions, we would try to find a plane that separated the points with maximum gap. We would need to compute some new amount that the points are above or below the plane. In geometric terms, if we are given  $x$  and  $y$  only, we somehow need to compute a  $z$  to work in that third dimension.

For a representation using  $n$  dimensions, we try to compute an  $n - 1$  separating *hyperplane*. We look at two and three dimensions in [chapter 4](#) and the general case in [chapter 5](#).



In this three-dimensional plot, I take the same values from the last two-dimensional version and lay the coordinate plane flat. I then add a vertical dimension. I push the red points below the plane and the blue ones above. With this construction, the coordinate plane itself separates the values.

While we can't separate the points in two dimensions, we can in three dimensions. This kind of mapping into a higher dimension is called the *kernel trick*. While the coordinate plane in this case might not be the ideal separating hyperplane, it gives you an idea of what we are

trying to accomplish. The benefit of *kernel functions* (as part of the similarly named “trick”) is that we can do far fewer explicit geometric computations than you might expect in these higher dimensional spaces.

It’s worth mentioning now that we don’t need to try quantum methods on small problems that are handled quite well using traditional means. We won’t see any kind of quantum advantage until the problems are big enough to overcome the quantum circuit overhead versus classical circuits. Also, if we come up with a quantum approach that can be simulated easily on a classical computer, we don’t really need a quantum computer.

A quantum computer with 1 qubit provides us with a two-dimensional working space. Every time we add a qubit, we double the number of dimensions. This is due to the properties of superposition and entanglement that I introduce in [chapter 7](#). For 10 qubits, we get  $2^{10} = 1024$  dimensions. Similarly, for 50 qubits we get  $2^{50} = 1,125,899,906,842,624$  dimensions.

Remember all those dimensions for the features and baseball players and films? We want to use a sufficiently large quantum computer to do the AI calculations in a *quantum feature space*. This is the main point: handle the extremely large number of dimensions coming out of the data in a large quantum feature space.

There is a quantum approach that can generate the separating hyperplane in the quantum feature space. There is another that skips the hyperplane step and produces a highly accurate classifying kernel function. As the ability to entangle more qubits increases, the successful classification rate improves as well. [8] This is an active area of research: how can we use entanglement, which does not exist classically, to find new or better patterns than we can do with strictly traditional methods?

#### **To learn more**

There are an increasing number of research papers being written connecting quantum computing with machine learning and other AI techniques. The results are somewhat fragmented. The best book pulling together the state of the art is Wittek. [19].

I do warn you again that quantum computers cannot process much data now!

For an advanced application of machine learning for quantum computing and chemistry, see Torial et al. [18]

## 1.5 Applications to financial services

Suppose we have a circle of radius 1 inscribed in a square, which therefore has sides of length 2 and area  $4 = 2 \times 2$ . What is the area  $A$  of the circle?

Before you try to remember your geometric area formulas, let's compute it a different way using ratios and some experiments.

Suppose we drop some number  $N$  of coins onto the square and count how many of them have their centers on or inside the circle. If  $C$  is this number, then

$$\frac{\text{area of the circle}}{\text{area of the enclosing square}} = \frac{A}{4} \approx \frac{C}{N} = \frac{\text{number of coins that land in the circle}}{\text{total number of coins}}.$$

So  $A \approx \frac{4C}{N}$ .

There is randomness involved here: it's possible they all land inside the circle or, less likely, outside the circle. For  $N = 1$ , we do not get an accurate estimate of  $A$  at all because  $\frac{C}{N}$  can only be 0 or 1.

### Question 1.5.1

If  $N = 2$ , what are the possible estimates of  $A$ ? What about if  $N = 3$ ?

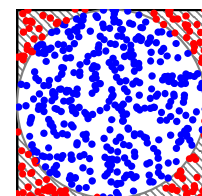
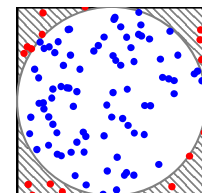
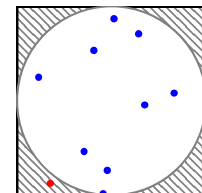
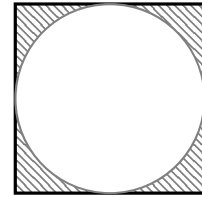
Clearly we will get a better estimate for  $A$  if we choose  $N$  large.

Using Python and its random number generator, I created 10 points whose centers lie inside the square. The plot on the right shows where they landed. In this case,  $C = 9$  and so  $A \approx 3.6$ .

For  $N = 100$  we get a more interesting plot with  $C = 84$  and  $A \approx 3.36$ . Remember, if different random numbers had been generated then this number would be different.

The final plot is for  $N = 500$ . Now  $C = 387$  and  $A \approx 3.096$ .

The real value of  $A$  is  $\pi \approx 3.1415926$ . This technique is called *Monte Carlo sampling* and goes back to the 1940s.



Using the same technique, here are approximations of  $A$  for increasingly large  $N$ . Remember, we are using random numbers and so these numbers will vary based on the sequence of values used.

|     |     |      |       |        |         |           |            |
|-----|-----|------|-------|--------|---------|-----------|------------|
| $N$ | 10  | 100  | 1,000 | 10,000 | 100,000 | 1,000,000 | 10,000,000 |
| $A$ | 3.6 | 3.36 | 3.148 | 3.1596 | 3.14336 | 3.141884  | 3.1414132  |

That's a lot of runs, the value of  $N$ , to get close to the real value of  $\pi$ . Nevertheless, this example demonstrates how we can use Monte Carlo sampling techniques to approximate the value of something when we may not have a formula. In this case we estimated  $A$ . For the example we ignored our knowledge that the formula for the area of a circle is  $\pi r^2$ , where  $r$  is the circle's radius.

In [section 6.7](#) we work through the math and show that if we want to estimate  $\pi$  within 0.00001 with probability at least 99.9999%, we need  $N \geq 82,863,028$ . That is, we need to use more than 82 million points! So it is possible to use a Monte Carlo method here but it is not efficient.

In this example, we know the answer ahead of time by other means. If we do not know the answer and do not have a nice neat formula to compute, Monte Carlo methods can be a useful tool. However, the very large number of samples needed to get decent accuracy makes the process computationally intensive. If we can reduce the sample count significantly, we can compute a more accurate result much faster.

Given that the title of this section mentions “finance,” I now note, perhaps unsurprisingly, that Monte Carlo methods are used in computational finance. The randomness we use to calculate  $\pi$  translates over into ideas like uncertainties. Uncertainties can then be related to probabilities, which can be used to calculate the risk and rate of return of investments.

Instead of looking at whether a point is inside or outside a circle, for the rate of return we might consider several factors that go into calculating the risk. For example,

- market size,
- share of market,
- selling price,
- fixed costs,
- operating costs,
- obsolescence,
- inflation or deflation,
- national monetary policy,
- weather, and
- political factors and election results.

For each of these or any other factor relevant to the particular investment, we quantify them and assign probabilities to the possible results. In a weighted way, we combine all possible

## 1.5. Applications to financial services

---

combinations to compute risk. This is a function that we cannot calculate all at once, but we can use Monte Carlo methods to estimate. Methods similar to, but more complicated than, the circle analysis in [section 6.7](#), give us how many samples we need to use to get a result within a desired accuracy.

In the circle example, even getting reasonable accuracy can require tens of millions of samples. For an investment risk analysis we might need many orders of magnitude greater. So what do we do?

We can and do use High Performance Computers (HPC). We can consider fewer possibilities for each factor. For example, we might vary the possible selling prices by larger amounts. We can consult better experts and get more accurate probabilities. This could increase the accuracy but not necessarily the computation time. We can take fewer samples and accept less precise results.

Or we might consider quantum variations and replacements for Monte Carlo methods. In 2015, Ashley Montanaro described a quadratic speedup using quantum computing. [\[12\]](#) How much improvement does this give us? Instead of the 82 million samples required for the circle calculation with the above accuracy, we could do it in something closer to 9,000 samples. ( $9055 \approx \sqrt{82000000}$ .)

In 2019, Stamatopoulos *et al* showed methods and considerations for pricing financial options using quantum computing systems. [\[16\]](#) I want to stress that to do this, we will need much larger, more accurate, and more powerful quantum computers than we have as of this writing. However, like much of the algorithmic work being done on industry use cases, we believe we are getting on the right path to solve significant problems significantly faster using quantum computation.

By using Monte Carlo methods we can vary our assumptions and do scenario analysis. If we can eventually use quantum computers to greatly reduce the number of samples, we can look at far more scenarios much faster.

### To learn more

David Hertz' original 1964 paper in the Harvard Business Review is a very readable introduction to Monte Carlo methods for risk analysis without ever using the phrase "Monte Carlo." [\[9\]](#) A more recent paper gives more of the history of these methods and applies them to marketing analytics. [\[6\]](#)

My goal with this book is to give you enough of an introduction to quantum computing so that you can read industry-specific quantum use cases and research papers. For example, to see modern quantum algorithmic approaches to risk analysis, see the articles by Woerner, Egger, et al. [20] [3]. Some early results on heuristics using quantum computation for transaction settlements are covered in Braine et al. [1]

## 1.6 What about cryptography?

You may have seen media headlines like

*Quantum Security Apocalypse!!!*

*Y2K??? Get ready for Q2K!!!*

*Quantum Computing Will Break All Internet Security!!!*

These breathless announcements are meant to grab your attention and frequently contain egregious errors about quantum computing and security. Let's look at the root of the concerns and insert some reality into the discussion.

RSA is a commonly used security protocol and it works something like this:

- You want to allow others to send you secure communications. This means you give them what they need to encrypt their messages before sending. You and only you can decrypt what they then give you.
- You publish a *public key* used to encrypt these messages intended for you. Anyone who has access to the key can use it.
- There is an additional key, your *private key*. You and only you have it. With it you can decrypt and read the encrypted messages. [15]

Though I phrased this in terms of messages sent to you, the scheme is adaptable for sending transaction and purchase data across the Internet, and storing information securely in a database.

Certainly if anyone steals your private key, there is a cybersecurity emergency. Quantum computing has nothing to do with physically taking your private key or convincing you to give it to a bad person.

What if I could compute your private key from the public key?

## 1.6. What about cryptography?

---

The public key for RSA looks like a pair of numbers  $(e, n)$  where  $n$  is a very larger integer that is the product of two primes. We'll call these primes numbers  $p$  and  $q$ . For example, if  $p = 982451653$  and  $q = 899809343$ , then  $n = pq = 884019176415193979$ .

Your private key looks like a pair of integers  $(d, n)$  using the very same  $n$  as in the public key. It is the  $d$  part you must really keep secret.

Here's the potential problem: if someone can quickly factor  $n$  into  $p$  and  $q$ , then they can compute  $d$ . That is, fast integer factorization leads to breaking RSA encryption.

Though multiplication is very easy and can be done using the method you learned early in your education, factoring can be very, very hard. For products of certain pairs of primes, factorization using known classical methods could take hundreds or thousands of **years**.

Given this, unless  $d$  is stolen or given away, you might feel pretty comfortable about security. Unless, that is, there is another way of factoring involving non-classical computers.

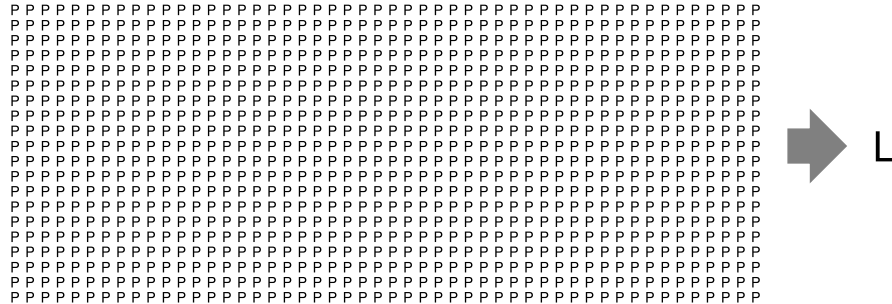
In 1995, Peter Shor published a quantum algorithm for integer factorization that is almost exponentially faster than known classical methods. We analyze Shor's algorithm in [section 10.6](#).

This sounds like a major problem! Here is where many of the articles about quantum computing and security start to go crazy. The key question is: **how powerful, and of what quality, must a quantum computing system be in order to perform this factorization?**

As I write this, scientists and engineers are building quantum computers with double digit numbers of *physical* qubits, hoping to get to triple digits in the next few years. For example, researchers have discussed qubit counts of 20, 53, 72, and 128. (Do note there is a difference between what people say they will have versus what they really have.) A physical qubit is the hardware implementation of the *logical* qubits we start discussing in [chapter 7](#).

Physical qubits have noise that cause errors in computation. Shor's algorithm requires fully fault-tolerant, error corrected logical qubits. This means we can detect and correct any errors that occur in the qubits. This happens today in the memory and data storage in your laptop and smartphone. We explore quantum error correction in [section 11.5](#).

As a rule of thumb, assume it will take 1,000 very good physical qubits to make one logical qubit. This estimate varies by researcher, degree of marketing hype, and wishful thinking, but I believe 1,000 is reasonable. We discuss the relationship between the two kinds of qubits in [chapter 11](#). In the meanwhile, we are in the Noisy Intermediate-Scale Quantum, or NISQ, era. The term NISQ was coined by physicist John Preskill in 2018. [14]



It will take many physical qubits to make one logical qubit

A further estimate is that it will take  $10^8 = 100$  million physical qubits to use Shor's algorithm to factor the values of  $n$  used in RSA today. That's approximately one hundred thousand logical qubits. On one hand, we have quantum computers with two or three digits worth of physical qubits. For Shor's algorithm to break RSA, we'll need eight digits worth. That's a huge difference.

These numbers may be too conservative, but I don't think by much. If anyone quotes you much smaller numbers, try to understand their motivation and what data they are using.

There's a good chance we won't get quantum computers this powerful until 2035 or much later. We may never get such powerful machines. Assuming we will, what should you do now?

First, you should start moving to so-called "post-quantum" or "quantum-proof" encryption protocols. These are being standardized at [NIST](#), the [National Institute of Standards and Technology](#), in the United States by an international team of researchers. These protocols can't be broken by quantum computing systems as RSA and some of the other classical protocols might be eventually.

You may think you have plenty of time to change over your transactional systems. How long will it take to do that? For financial institutions, it can take ten years or more to implement new security technology.

Of greater immediate importance is your data. Will it be a problem if someone can crack your database security in 15, 30, or 50 years? For most organizations the answer is a loud YES. Start looking at hardware and software encryption support for your data using the new post-quantum security standards now.

Finally, quantum or no quantum, if you do not have good cybersecurity and encryption strategies and implementations in place now, you are exposed. Fix them. Listen to the people who make quantum computing systems to get a good idea of if and when they might be used to break encryption schemes. All others are dealing with second- and third-hand knowledge.

### To learn more

Estimates for when and if quantum computing may pose a cybersecurity threat vary significantly. Any study on the topic will necessarily need to be updated as the technology evolves. The most complete analysis as of the time this book was first published appears to be Mosca and Piani. [13]

## 1.7 Summary

In this first chapter we looked at what is motivating the recent interest in quantum computers. The lone 1s and 0s of classical computing bits are extended and complemented by the infinite states of qubits, also known as quantum bits. The properties of superposition and entanglement give us access to many dimensions of working memory that are unavailable to classical computers.

Industry use cases for quantum computing are nascent but the areas where experts believe it will be applicable sooner are chemistry, materials science, and financial services. AI is another area where quantum may boost performance for some kinds of calculations.

There has been confusion in traditional and social media about the interplay of security, information encryption, and quantum computing. The major areas of misunderstanding are the necessary performance requirements and the timeline.

In the next chapter, we look at classical bit-based computing to more precisely and technically explore how quantum computing may help us attack problems that are otherwise impossible today. In [chapter 3](#) through [chapter 6](#) we work through the mathematics necessary for you to see how quantum computing works. There is a lot to cover, but it is worth it to be able to go deeper than a merely superficial understanding of the “whats,” “hows,” and “whys” of quantum computing.

## References

- [1] Lee Braine et al. *Quantum Algorithms for Mixed Binary Optimization applied to Transaction Settlement*. 2019. URL: <https://arxiv.org/abs/1910.05788>.
- [2] Yudong Cao et al. “Quantum Chemistry in the Age of Quantum Computing”. In: *Chemical Reviews* (2019).
- [3] Daniel J. Egger et al. *Credit Risk Analysis using Quantum Computers*. 2019. URL: <https://arxiv.org/abs/1907.03044>.

- [4] FermiLab. *What is the number of atoms in the world?* 2014. URL: <https://www.fnal.gov/pub/science/inquiring/questions/atoms.html>.
- [5] Richard P. Feynman. “Simulating Physics with Computers”. In: *International Journal of Theoretical Physics* 21.6 (June 1, 1982), pp. 467–488.
- [6] Peter Furness. “Applications of Monte Carlo Simulation in marketing analytics”. In: *Journal of Direct, Data and Digital Marketing Practice* 13 (2 Oct. 27, 2011).
- [7] Aram W. Harrow, Avinatan Hassidim, and Seth Lloyd. “Quantum Algorithm for Linear Systems of Equations”. In: *Physical Review Letters* 103 (15 Oct. 2009), p. 150502.
- [8] Vojtěch Havlíček et al. “Supervised learning with quantum-enhanced feature spaces”. In: *Nature* 567.7747 (Mar. 1, 2019), pp. 209–212.
- [9] David B Hertz. “Risk Analysis in Capital Investment”. In: *Harvard Business Review* (Sept. 1979).
- [10] Abhinav Kandala et al. “Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets”. In: *Nature* 549 (Sept. 13, 2017), pp. 242–247.
- [11] Rachel Link. *How Much Caffeine Do Coke and Diet Coke Contain?* 2018. URL: <https://www.healthline.com/nutrition/caffeine-in-coke>.
- [12] Ashley Montanaro. “Quantum speedup of Monte Carlo methods”. In: *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences* 471.2181 (2015), p. 20150301.
- [13] Michele Mosca and Marco Piani. *Quantum Threat Timeline*. 2019. URL: <https://globalriskinstitute.org/publications/quantum-threat-timeline/>.
- [14] John Preskill. *Quantum Computing in the NISQ era and beyond*. URL: <https://arxiv.org/abs/1801.00862>.
- [15] R. L. Rivest, A. Shamir, and L. Adleman. “A Method for Obtaining Digital Signatures and Public-key Cryptosystems”. In: *Commun. ACM* 21.2 (Feb. 1978), pp. 120–126.
- [16] Nikitas Stamatopoulos et al. *Option Pricing using Quantum Computers*. 2019. URL: <https://arxiv.org/abs/1905.02666>.
- [17] Ewin Tang. *A quantum-inspired classical algorithm for recommendation systems*. 2019. URL: <https://arxiv.org/pdf/1807.04271.pdf>.
- [18] Giacomo Torlai et al. *Precise measurement of quantum observables with neural-network estimators*. URL: <https://arxiv.org/abs/1910.07596>.
- [19] P. Wittek. *Quantum Machine Learning. What quantum computing means to data mining*. Elsevier Science, 2016.
- [20] Stefan Woerner and Daniel J. Egger. “Quantum risk analysis”. In: *npj Quantum Information* 5 (1 Feb. 8, 2019), pp. 198–201.